

Language Server Protocol para YAP

Programação em Lógica
CC3012

2026

Docente: Vitor Costa

André Mendes up202307449
Eurico Magalhães up200701520

Conteúdos

Motivação	3
Language Server Protocol	3
pygls	4
Visual Studio Code ou Emacs	5
Configuração do sistema	5

Motivação

Um dos possíveis trabalhos apresentados pelo professor foi o desenvolver uma solução para o seguinte problema: os utilizadores de editores de texto e IDE's esperam que estes programas possuam certas funcionalidades, como coloração de texto consoante a sintaxe da linguagem de programação, verificação e validação dessa sintaxe, procura de símbolos e definições, entre outras.

O compilador de Prolog YAP ainda não tem uma integração com possíveis soluções para este problema. Assim, este trabalho pretende investigar e implementar uma solução, usando o Language Server Protocol.

Language Server Protocol

O Language Server Protocol¹ (LSP) é um protocolo de comunicação entre editores de texto e IDE's, e um servidor de uma linguagem de programação, que providencia ao editor as regras e instruções necessárias à implementação de funcionalidades de formatação e uso dessa linguagem no contexto do editor ou IDE.

Desenvolvido pela Microsoft, e *open source*, é um protocolo usado no Visual Studio Code² (VSCode) e, como este editor é bastante usado, atualmente, pelos alunos de Ciência de Computadores, o professor decidiu trabalhar neste contexto.

A motivação por trás do LSP, como explica a Microsoft, é a seguinte: em vez de cada editor de texto ter que implementar todas as linguagens de programação que pretende suportar, porque não ter um protocolo de comunicação que, quando implementado uma vez em cada editor, permite reconhecer todas as linguagens que implementem, por sua vez, esse protocolo.

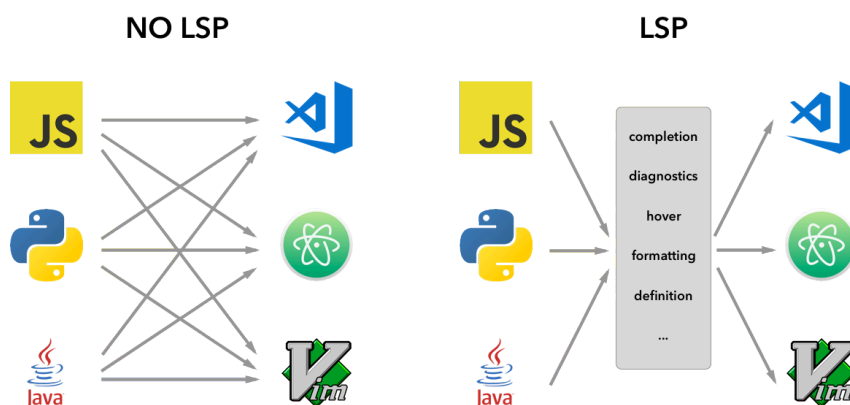


Figura 2: Motivação do LSP (fonte: site da API do VSCode²)

¹<https://microsoft.github.io/language-server-protocol/>

²<https://code.visualstudio.com/api/language-extensions/language-server-extension-guide>

Este protocolo implementa mensagens codificadas em JSON RPC³ que são trocadas entre cliente e servidor.

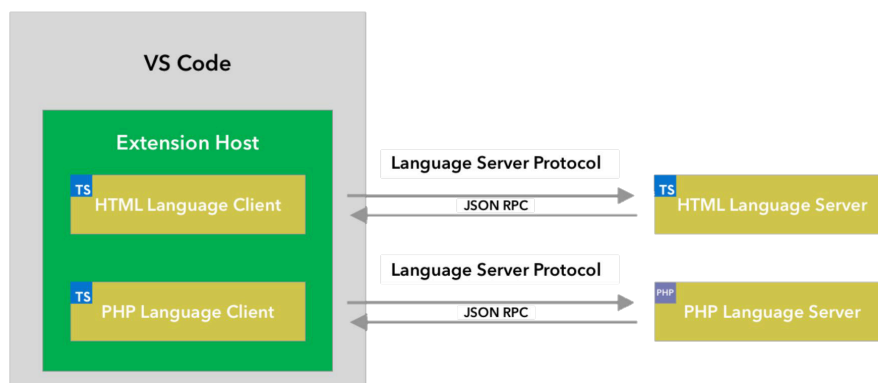


Figura 3: Funcionamento do LSP (fonte: site da API do VSCode²)

Como este protocolo se tornou popular, foram desenvolvidas várias ferramentas que permitem mais facilmente implementar um cliente/servidor LSP, evitando assim termos que, por exemplo, implementar todas as mensagens JSON.

Uma destas ferramentas é o pygls⁴.

pygls

O pygls é uma “implementação genérica do LSP escrita em Python”⁴. Usando o pygls deve ser possível escrever um servidor LSP em Python, de uma maneira relativamente simples.

Por exemplo, o código seguinte recebe um documento e, em cada linha que acabe em “hello.”, retorna ao editor de texto a hipótese de autocompletar com “world” ou “friend” (linhas 18-20).

Esta sugestão de autocompletar só acontece quando o utilizador insere um “.” (linha 9).

³<https://microsoft.github.io/language-server-protocol/specifications/lsp/3.17/specification/>

⁴<https://pygls.readthedocs.io/en/latest/>

```

1 from pygls.lsp.server import LanguageServer
2 from lsprotocol import types
3
4 server = LanguageServer("example-server", "v0.1")
5
6
7 @server.feature(
8     types.TEXT_DOCUMENT_COMPLETION,
9     types.CompletionOptions(trigger_characters=["."]),
10 )
11 def completions(params: types.CompletionParams):
12     document =
13         server.workspace.get_text_document(params.text_document.uri)
14     current_line = document.lines[params.position.line].strip()
15
16     if not current_line.endswith("hello."):
17         return []
18
19     return [
20         types.CompletionItem(label="world"),
21         types.CompletionItem(label="friend"),
22     ]
23
24 if __name__ == "__main__":
25     server.start_io()

```

O cliente dependerá, claro, do editor de texto/IDE. É aqui que surge o primeiro impedimento ao objetivo do trabalho.

Visual Studio Code ou Emacs

O VSCode⁵, nas palavras do professor, pode ser “bastante burocrático” na implementação de um cliente LSP. De facto, esse cliente teria de ser um projecto implementado em TypeScript, e o *debugging* envolve também muita burocracia, especialmente quando comparado com a alternativa: Emacs.

O Emacs⁶ é muito personalizável e, crucialmente, tem uma extensão que permite trabalhar com LSP, Eglot⁷. É relativamente fácil fazer com que esta extensão lance o nosso servidor, como veremos no próximo capítulo. Permite também monitorizar as mensagens enviadas entre Emacs e o servidor de uma maneira simples.

Configuração do sistema

⁵<https://code.visualstudio.com/>

⁶<https://www.gnu.org/software/emacs/>

⁷<https://joaotavora.github.io/eglot/>